广告素材合规审核 • 模型实验报告

数据：HuggingFace zjunlp/ChineseHarm-bench ｜ 测试集 956 条 ｜ 2026-10-15 ｜ 硬件：RTX 3050 Ti Laptop (4GB)

一、任务与数据

任务定义：广告素材投放前合规审核。输入一段文案，输出 6 类判定；模型没把握时输出「需人工复核」（置信度阈值拒判）。

- **类别**：0 不违规 / 1 黑产广告 / 2 欺诈 / 3 博彩 / 4 谩骂引战 / 5 低俗色情
- **规模**：去重后 9,503 条，分层抽样切分 train 7,600 / dev 947 / test 956
- **测试集分布不均**：低俗色情 201、谩骂引战 191、黑产广告 188、博彩 176、不违规 100、**欺诈仅 100**
- **文本长度**：中位数 19 字、平均 42 字 → max_len=64

二、最终模型核心指标

测试集准确率	macro-F1	欺诈类召回	量化后体积
93.1%	**93.4%**	**94%**	**146 MB**
基线 93.3%（持平）	**基线 93.0%（+0.4）**	**基线 77%（+17 个点）**	**390 MB → 压缩 63%**

三、四套方案横向对比

方案	关键配置	accuracy	macro-F1	单条推理(CPU)
BERT 微调 + 类别加权（最终采用）	batch 8 / max_len 64 / lr 2e-5 / 3 epoch / 加权损失	0.931	**0.934**	103 ms（量化后 73 ms）
BERT 微调（无加权基线）	同上，普通 CrossEntropy	0.933	0.930	103 ms
BERT 动态量化 (int8)	对基线模型 quantize_dynamic	0.915	0.915	73 ms
fastText 字级 · 手动参数	dim 32 / wordNgrams 2 / epoch 100	—	0.908	<1 ms
fastText 字级 · 自动调参	autotune 30s（结果有波动）	—	0.894~0.909	<1 ms
fastText 词级 · 手动参数	jieba 分词	—	0.872	<1 ms
fastText 词级 · 自动调参	体积上限 50M	—	0.855	<1 ms
随机森林 + TF-IDF	自带 8:2 切分（非同一测试集）	0.784	0.780	<1 ms

选型结论：BERT 比 fastText 字级高约 2.6 个点 F1，但推理慢百倍。若追求吞吐，fastText 字级 + 置信度兜底也是可用的粗筛方案；本场景误判代价高，最终采用 BERT（量化版粗筛 + 原始版复核的分层部署）。

四、类别加权实验：把欺诈召回从 77% 拉到 94%

问题：欺诈是少数类（测试集仅 100 条），无加权基线 26 条误判（11 条→黑产广告、9 条→博彩），召回仅 0.77。

方法：加权 CrossEntropyLoss，weight = √(N / (K·count))，开根号缓和过度补偿。实测权重：不违规 1.27、欺诈 1.26、黑产广告 0.92、博彩 0.95。

类别	F1 · 基线	F1 · 加权	召回 · 基线	召回 · 加权	评价
欺诈	0.851	**0.904**	0.77	**0.94**	核心目标达成
不违规	0.985	0.990	—	0.99	持平
黑产广告	0.950	0.963	—	0.96	小幅提升
博彩	0.940	0.941	—	0.92	持平
谩骂引战	0.934	**0.912**	—	0.87	略有牺牲
低俗色情	0.920	**0.895**	—	0.94	精确率下降（谩骂→低俗 24 条）

业务判断：审核场景漏放欺诈的代价远高于多审几条低俗色情，这笔交换划算。**但要清醒：**诊断发现约一半误判样本属于「标注边界模糊」——「上岸」「打米」「七位数」是博彩黑话，「薇/丝信/柏」是防审核变形字，欺诈与博彩引流的语料本身犬牙交错。加权只能救回样本不均衡的部分，剩下的天花板在标注质量，继续调参收益有限。

五、最终模型混淆矩阵（测试集 956 条）

真实 \ 预测	不违规	黑产广告	欺诈	博彩	谩骂引战	低俗色情
不违规 (100)	99	0	0	0	0	1
黑产广告 (188)	0	181	**6**	0	0	1
欺诈 (100)	0	**5**	94	1	0	0
博彩 (176)	0	2	**7**	161	0	6
谩骂引战 (191)	1	0	0	0	166	**24**
低俗色情 (201)	0	0	1	4	**7**	189

绿=判对，红=主要混淆源。最刺眼的是谩骂引战→低俗色情 24 条：辱骂文本常带性羞辱词，两类语义天然交叠。

六、置信度阈值拒判（业务驱动的改进）

原架构 argmax 永远硬判一类，但真实审核中「没把握」本身是合法答案。softmax 最高置信度低于阈值 → 转人工复核。

阈值	自动判定占比	自动判定准确率	违规召回	说明
0.00	100.0%	93.10%	92.4%	纯 argmax
0.60（采用）	96.0%	94.88%	90.2%	4% 流量进人工，准确率 +1.8
0.80	90.9%	96.55%	86.4%	更保守
0.90	85.7%	97.92%	82.1%	召回损失过大

七、动态量化收益（CPU 部署视角）

体积

390 → 146 MB

压缩到 37.3%

单条推理

103 → 73 ms

1.41x

准确率

93.3 → 91.5%

-1.8 个点

部署建议：量化版做线上粗筛（吞吐高、体积极轻），低置信度或高危命中回落到原始模型 / 人工。注意坑：修好 device 自动识别后，量化脚本必须把 `bert_train` 模块级 config 和本地 config 的 device **都**强制为 cpu，否则数据在 cuda、模型在 cpu 直接报错。

八、知识蒸馏：12 层 Teacher → 4 层 Student

Student 从哪来：本机只有 `bert-base-chinese` 一个预训练权重，没有现成的小中文 BERT，也不方便联网下载。所以采用**层裁剪初始化**——从 Teacher 的 12 层里按 uniform 策略等间隔抽第 [0, 4, 7, 11] 层，embedding / pooler / 分类头整块搬过来，再用蒸馏微调。**只抽层、还没蒸馏时 macro-F1 只有 0.2445**，蒸馏把它拉到 0.9230（**+0.679**）——这一栏最能说明蒸馏到底在干什么。

参数量

102.3 → 45.6 M

压缩 2.24x

单条推理

5.19 → 1.68 ms

3.08x

accuracy

93.1 → 92.5%

-0.6 个点

macro-F1

0.9343 → 0.9230

-1.1 个点

与 Teacher 一致率

92.6%

行为高度对齐

类别	Teacher F1	Student F1	差值	说明
不违规	0.9900	0.9746	**-0.015**	最简单的一类，几乎无损
黑产广告	0.9628	0.9600	-0.003	完全保住
谩骂引战	0.9121	0.9286	**+0.017**	小模型反而更好
低俗色情	0.8957	0.9005	**+0.005**	基本持平
博彩	0.9415	0.9231	**-0.018**	轻微损失
欺诈	0.9038	0.8513	**-0.053**	**召回 0.94 → 0.83，容量不足时少数类最先掉**

踩过的坑：alpha 别照抄论文的 0.5。第一轮用 `alpha=0.5 / 3 epochs`，Student 的欺诈召回只有 **0.77**，而 Teacher 加权后是 0.94——上一步类别加权换来的收益被蒸馏弄丢了。原因是软损失（KL）本身**不带类别权重**，它占一半话语权时就把少数类的判别拉平了。把 alpha 提到 **0.7**、epochs 提到 5 之后，硬损失（带类别权重）重新主导，欺诈召回回到 0.83：

配置	accuracy	macro-F1	欺诈召回
alpha=0.5, 3 epochs	0.9153	0.9133	0.77
alpha=0.7, 5 epochs（最终采用）	**0.9247**	**0.9230**	**0.83**

另外验证集 F1 在第 4 轮达峰（0.9257）后第 5 轮回落到 0.9160，`best_f1` 存盘策略正确保存了第 4 轮而非最后一轮。

九、非结构化剪枝：50% 稀疏度

方法：全局 L1 幅度剪枝（magnitude pruning），分 3 步渐进到 50%，每步剪完立刻微调 1 轮让网络自我修复。只剪 Transformer 内的 Linear，**跳过分类头和 pooler**——它们参数量极小（768×6 / 768×768），剪了省不了空间却直接伤输

出层。

	原始	剪枝 50%	剪枝 + 量化
accuracy	0.9310	**0.9320**	**0.9163**
macro-F1	0.9343	0.9305	**0.9146**
稀疏度	0%	50.0%	50.0%
存盘体积	390.2 MB	390.2 MB	**145.6 MB**
gzip 后	362.3 MB	**244.3 MB**	**98.0 MB**
CPU 单条	63.6 ms	49.7 ms	56.6 ms

关键认知：非结构化剪枝省的是「体积」，不是「速度」。稀疏度已经 50%，但存盘体积**一点没变**（390.2 MB）——因为权重只是被置 0，用 float32 稠密格式存盘时 0 和别的数占一样大的空间。真正的收益要到 gzip 之后才显现（-32.6%，叠加量化后 -72.9%）。CPU 耗时也几乎没变化，因为普通稠密算子不会跳过 0，**必须配合稀疏计算库或专用硬件才有加速。**

稀疏度阶梯（验证集，用于挑选精度/体积平衡点）：

目标稀疏度	实际稀疏度	val_acc	val_f1
17%	16.67%	0.9407	0.9414
33%	33.33%	0.9322	0.9330
50%	50.00%	0.9396	0.9390

未剪枝基线的验证 F1 是 **0.9441**，所以剪枝是**有代价**的，只是代价很小：17% 时 -0.3 个点、33% 时 -1.1 个点、50% 时 -0.5 个点。曲线非单调（33% 反而最低）是因为每步只微调了 1 轮、修复程度不同。关键点在于：**剪掉一半权重，验证 F1 只掉 0.5 个点**，说明这个 12 层模型在广告审核任务上明显过参数化，冗余度很高。

踩过的坑：别用 PyTorch 的 `prune.global_unstructured` **做迭代剪枝。**它给每个 Linear 挂上 `weight_orig` + `weight_mask` 两份 85M 的全精度缓冲，显存凭空多出约 0.68 GB。本机只有 4 GB 且被浏览器等常驻程序分走一部分，实测直接触发 WDDM 换页，训练速度从 6 it/s 掉到 **0.05 it/s（20 秒一个 batch）**。改成手动维护掩码（阈值在 CPU 上算，原地置 0，每个 `optimizer.step()` 后重新压制）后恢复 **5.7 it/s**，快了 100 倍。
另外它的 `amount` 是「在当前尚未被剪的权重中再剪掉的比例」而非累计比例，实测直接传 [1/6, 2/6, 3/6] 会复利叠加成 16.67% → 44.44% → 72.22%，第三步远超计划的 50%。

十、本次改造修复的工程问题（10 个）

- `config.device` 被二次赋值硬编码成 cpu → 有显卡也永远跑 CPU；拆出 `quant_device` 修复
- `train()` 无 return 且指标只在最优轮追加 → 训练曲线只有 1 个点甚至 None
- test_dataloader `drop_last=True` 丢尾部样本 → 测试指标不可信
- `lr=2e-4` 与注释 2e-5~5e-5 不符 → 回归 2e-5
- `rf_train.py` 被 IDE 误插入 `from pygments.lexer import words` → 启动即崩
- **fastText 打不开中文路径**（C++ 窄字符内核，路径含「深度学习」必崩）→ 加 ASCII 临时目录兼容层
- fastText 自动调参挑到大 dim，输入矩阵 11GB 撑爆内存且无 Python 异常 → 加体积上限
- 量化脚本 device 撞车（本报告第七节）
- **迭代剪枝用 PyTorch prune 缓冲**导致显存膨胀、触发 WDDM 换页，速度掉 100 倍 → 改手动掩码（本报告第九节）
- **蒸馏 alpha=0.5 让不带类别权重的 KL 稀释硬损失**，欺诈召回从 0.94 回退到 0.77 → alpha 提到 0.7（本报告第八节）

十一、遗留与下一步

- **谩骂 ↔ 低俗色情边界**（24 条混淆）：可引入二阶段细分类或更换标注口径
- **黑话变形字**（米/柏/薇/丝信）：fastText 字级天然占优；BERT 可加对抗增强或黑话词典
- **DeepSeek 对照实验**：提示词已就绪（6 类定义 + 消歧规则 + 12 条 Few-Shot + 拒判），配置 DEEPSEEK_API_KEY 后可量化回答「这个场景值不值得上大模型」
- **欺诈标注质量**：建议抽 100 条欺诈/博彩边界样本人工复核，是比调参更划算的投入
- **Student 的欺诈召回仍是 0.83**（Teacher 0.94）：可尝试 6 层 Student、更高 alpha，或只对非欺诈类做蒸馏、欺诈类保留 Teacher
- **结构化剪枝**：本节是非结构化（只置 0），若想真正拿到推理加速，应做 attention head / FFN 维度的结构化裁剪，再配合稀疏算子